

Supplementary Material

GIST: Goals as Implicit Supervision at Training for Visuomotor Policy Learning

APPENDIX I

THE UNCERTAINTY-AWARE HIERARCHICAL POLICY

This appendix describes the uncertainty-aware hierarchical policy that we use as the strongest hierarchical baseline in the main paper. It consists of a high-level policy π_{hi} that predicts a distribution over the next sub-goal and a low-level policy π_{lo} that consumes that distribution.

A. GMM-Based High-Level Policy

π_{hi} predicts the next sub-goal gripper pose $g(t) \in \mathbb{R}^{K \times 3}$ from the current scene point cloud $P_t = \{\mathbf{p}_i\}_{i=1}^N$, unprojected from o_t as described in the problem statement, and the current gripper keypoints e_t . It outputs the same per-anchor Gaussian mixture as the auxiliary head of π_{aux} (Section IV of the main paper), here with one component on every input point, so that multi-modal sub-goals are represented explicitly rather than averaged away. π_{hi} uses no images and no language; its only inputs are 3D points.

a) Architecture and Loss: The K current gripper keypoints are prepended to the scene cloud with a binary indicator channel, giving $N+K$ input points. A PointNet++ segmentation network [1] (six set-abstraction and six feature-propagation levels; details in Table II) yields a per-point feature, from which a shared head emits a displacement $\Delta_n \in \mathbb{R}^{K \times 3}$ and a logit ℓ_n : every input point, scene and gripper alike, is an anchor that proposes a sub-goal relative to itself and votes on its plausibility. These predictions define exactly the Gaussian mixture of Section IV of the main paper, with the $N+K$ points as anchors, and π_{hi} is trained with the same negative log-likelihood objective, variance ladder and uniform-weight term; we denote this loss $\mathcal{L}_{hi}$.

b) Training and Inference: π_{hi} is trained on $\mathcal{D}_{robot}$ with $\mathcal{L}_{hi}$ alone. Since the target jumps at boundaries, uniform frame sampling makes the model hedge between the outgoing and incoming sub-goal; we counter this with *transition-focused sampling* (frames within $\pm r$ steps of a boundary receive a fixed fraction $p_{tr}=0.5$ of the sampling probability) and *label swapping* (at distance $\delta \leq r$ from a boundary the target is swapped across the boundary with probability $p_{max}(1 - \delta/r)$; $r=5$, $p_{max}=0.5$), so the mixture learns to place mass on both sub-goals near a transition, the multi-modality π_{lo} 's weighted cross-attention consumes. At inference a single forward pass yields the mixture; the gripper anchors are dropped, the weights renormalised over the N scene anchors, and π_{lo} retains the top- J candidates.

The same procedure, run offline on the demonstration frames, produces the mixtures π_{lo} trains on.

B. Uncertainty-Aware Low-Level Policy

π_{lo} shares its backbone with π_{aux} (Section IV of the main paper): the same frozen DINOv2 patch tokens with world-frame anchors, the same current-gripper tokens, the same RoPE4D trunk producing grounded visual tokens $\tilde{z}(o_t)$, and the same flow-matching DiT generating action chunks. It differs in how the goal enters. Rather than serving as an auxiliary target, the goal is now an *input*: π_{hi} (Sec. I-A) provides a Gaussian mixture over candidate goal poses $\{(\hat{e}_j, w_j)\}_{j=1}^N$, $\hat{e}_j \in \mathbb{R}^{K \times 3}$, $\sum_j w_j = 1$, of which we retain the J highest-weight candidates per observation step. The auxiliary goal head of π_{aux} is removed. The retained candidates reach the policy through two routes, one geometric and one direct.

a) Goal Candidates as Grounded Tokens.: Each retained candidate is MLP-encoded from its flattened keypoints and appended to the trunk sequence, RoPE-anchored at the keypoint of $\hat{e}_j$ coinciding with the end-effector origin and at time $\tau = 1$, the end of the action horizon, which places it strictly after every observation token and marks it as a future target. The mixture weights bias attention to the goal keys: for a query q and goal key k_j ,

$$\text{logit}(q, k_j) = q^\top k_j / \sqrt{d} + \log(w_j / w_{\max}), \quad (1)$$

with no bias on patch or gripper keys. Normalising by the largest retained weight $w_{\max}$ is necessary because this softmax also spans the scene tokens: a raw $\log w_j$ bias would suppress all goal keys exactly when π_{hi} is uncertain. As in π_{aux} , only patch tokens leave the trunk, so this route informs the DiT purely by letting visual tokens relate themselves, at true metric offset, to where the robot is being sent.

b) Weighted Cross-Attention to Goals.: Independently, a second MLP encodes the same J candidates into goal tokens $\{c_j\}$ for the DiT. In every cross-attention block, before the visual cross-attention, the state and action tokens attend to them with the mixture weights as a log-prior,

$$\text{logit}_{ij} = q_i^\top k_j / \sqrt{d} + \log w_j, \quad (2)$$

i.e. un-normalised scores scaled by w_j : likely candidates attract attention by default while content can re-rank among them¹. The output joins the residual stream and the block

¹Weights are clamped at 10^{-8} before the logarithm, masking zero-weight slots.

proceeds as before. Since this softmax spans goal keys only, no $w_{\max}$ normalisation is needed.

c) **Training.**: π_{lo} is trained on $\mathcal{D}_{\text{robot}}$ with the flow-matching objective $\mathcal{L}_{\text{FM}}$ of Section IV of the main paper alone; there is no auxiliary loss. The goal mixture fed to π_{lo} during training is the one predicted by π_{hi} on the demonstration frames, so that π_{lo} learns to consume the distribution it will see at deployment, including its errors and ambiguity. To also expose it to clean goals, with probability p_{gt} per sample we replace the predicted mixture by a sparse ground-truth mixture: a single mode holding the demonstration goal $g(t)$ with weight 1, or, within $\pm r$ frames of a sub-task boundary, two modes holding the current and the adjacent goal with triangular weights $w_{\text{adj}} = p_{\max}(1 - \frac{\delta}{r+1})$, $w_{\text{cur}} = 1 - w_{\text{adj}}$, where δ is the distance to the boundary. All remaining candidate slots receive zero weight. We use $p_{gt} = 0.5$, $r = 5$, $p_{\max} = 0.5$; validation always uses the predicted mixture. We train π_{hi} and π_{lo} independently. Hyperparameters not listed in Table III are as for π_{aux} .

d) **Inference.**: π_{hi} predicts the mixture once per observation; the top- J candidates, grounded tokens and goal encodings are likewise computed once, and the DiT integrates S Euler steps to generate $a_{t:t+H}$, of which the first H_{exec} actions are executed.

APPENDIX II

SUB-GOAL EXTRACTION BY BAYESIAN CHANGEPOINT DETECTION

A. Feature

The extraction procedure we propose treats the demonstration as a sequence of piecewise-stationary segments and places a sub-goal boundary wherever the segment changes. Its only feature is the end-effector speed. Velocities are obtained by central finite differences of the recorded end-effector positions, deliberately ignoring the simulator's own velocities so that the same statistic is available from joint encoders on a real robot, and the speed is the norm of this difference, z-scored over the trajectory with no smoothing. We write $u_t \in \mathbb{R}$ for the z-scored speed at frame t .

B. Run-Length Recursion

Bayesian online changepoint detection [2] never thresholds the feature. It maintains a posterior over the run length r_t , the number of frames elapsed since the most recent boundary, and calls a frame a boundary when that posterior concentrates on $r_t = 0$. Conditioned on a run length the frames of the current segment are draws from one segment model, so the recursion is carried on the joint $P_t(r) \triangleq P(r_t = r, u_{1:t})$, for which only two transitions are possible at each frame. The run grows by one, or it resets and a new segment begins,

$$P_t(r+1) = P_{t-1}(r) \ell_t^{(r)} (1 - H_t), \quad (3)$$

$$P_t(0) = \sum_r P_{t-1}(r) \ell_t^{(r)} H_t, \quad (4)$$

where $\ell_t^{(r)} = p(u_t \mid u_{t-r:t-1})$ is the probability the run of length r assigns to the incoming frame and H_t is the hazard,

the prior probability that a segment ends at t . We take the hazard to be constant after a warm-up and zero before it,

$$H_t = \begin{cases} 0 & t < \tau_{\min}, \\ 1/\lambda & t \geq \tau_{\min}, \end{cases} \quad (5)$$

so λ is the expected segment length in frames, while $\tau_{\min}$ suppresses boundaries during the opening reach, where the arm is accelerating from rest and the segment statistics are not yet informative. Normalising P_t over r gives the run-length posterior, and the changepoint probability we read out at frame t is $c_t = P(r_t = 0 \mid u_{1:t})$. Run lengths are truncated at $r_{\max} = 200$ frames and hypotheses whose log probability falls below -23 are pruned, which bounds the cost at $\mathcal{O}(Tr_{\max})$ for a demonstration of T frames.

C. Segment Model

Within a segment the speed is modelled as Gaussian with unknown mean and variance, under the conjugate normal-inverse-Wishart prior on that pair, which for our one-dimensional feature reduces to a normal-inverse-gamma. Because the feature is z-scored, the prior mean $m_0 = 0$ and prior scale $\Psi_0 = 1$ are already matched to it, and what remains is one pseudo-observation of confidence in the mean, $\kappa_0 = 1$, and $\nu_0 = 3$ degrees of freedom, the weakly informative choice $d+2$ at $d = 1$. Conjugacy is what keeps the recursion tractable. A run that has absorbed n frames with sample mean $\bar{u}$ and centred sum of squares S has posterior

$$\kappa_n = \kappa_0 + n, \quad \nu_n = \nu_0 + n, \quad m_n = \frac{\kappa_0 m_0 + n \bar{u}}{\kappa_n}, \quad (6)$$

$$\Psi_n = \Psi_0 + S + \frac{\kappa_0 n}{\kappa_n} (\bar{u} - m_0)^2, \quad (7)$$

and integrating the mean and the variance out of the Gaussian leaves a Student-t predictive in closed form,

$$\ell_t^{(n)} = \text{St} \left(u_t \mid m_n, \frac{\Psi_n(\kappa_n + 1)}{\kappa_n \nu_n}, \nu_n \right), \quad (8)$$

with location m_n , squared scale as given, and ν_n degrees of freedom. The heavy tails carry weight here, because a young run has seen few frames and should not be confident enough to read a single fast frame as a changepoint. The sufficient statistics are carried forward by Welford updates, so extending a run costs constant time and nothing is recomputed from the raw frames. No quantity above is learned from data, and the extractor makes one pass over each demonstration.

D. Boundaries

Peaks of c_t that clear a fixed height and are separated by a minimum spacing become boundaries, the spacing being what suppresses the cluster of adjacent frames that a single transition produces. Where the task has a gripper, open and close events are added as boundaries alongside the detected ones, while Push-T has none, so its boundaries are speed changepoints alone. Boundaries inside the warm-up are dropped, the final frame is always a boundary, and the sub-goal at each frame is the gripper pose at the next boundary. Settings are listed in Table II.

TABLE I
HYPERPARAMETERS OF π_{aux} .

Observation	
Cameras C / observation steps T_o	2 / 2
Image resolution / random crop	256×256 / 224×224
Proprioceptive state s_t	10-D (EE position, 6-D rotation, gripper)
Gripper keypoints K	4
Grounded encoder	
Backbone	DINOv2-base (frozen), patch size 14
Patch tokens per image N_p	256
Token width	1024
RoPE4D blocks L / heads / head dim	2 / 16 / 64
Metric scale λ / time scale / RoPE base	5.0 / 1.0 / 100
Goal head	
MLP hidden width	512
Anchors per observation step N_a	516 ($4 + 2 \times 256$)
Variance ladder Σ	{0.01, 0.05, 0.1, 0.25, 0.5}
Uniform-weight coefficient λ_u	0.1
Auxiliary weight c_1	0.1
DiT	
Blocks / width / heads	12 (alternating cross/self) / 1024 / 16
Dropout	0.1
Action chunk H / executed H_{exec}	16 / 8
Action dimension	10
Flow-time schedule	$\alpha = (s - b)/s$, $b \sim \text{Beta}(1.5, 1)$, $s = 0.999$
Inference Euler steps S	10
Optimisation	
Optimiser	AdamW, lr 10^{-4} , betas (0.95, 0.999), wd 10^{-6}
LR schedule	cosine, 500 warm-up steps
Batch size / epochs	128 / 100
EMA	power 0.75, max decay 0.9999
Demonstrations	100

APPENDIX III HYPERPARAMETERS

Tables I, II and III list the settings of π_{aux} , π_{hi} and π_{lo} . Values are those used for the MimicGen tasks. The only quantities that differ elsewhere are the number of demonstrations and the number of epochs, which are 206 and 200 for Push-T against 100 and 100 for the MimicGen tasks. The trunk goal prior of π_{lo} is written in the table as three coefficients on the content term, the relative log-weight and the overall log-weight scale. The values used reproduce exactly the attention bias of Appendix I, in which likely candidates are ranked against one another without the goal keys being suppressed as a group when π_{hi} is uncertain.

TABLE II
HYPERPARAMETERS OF THE HIGH-LEVEL POLICY π_{hi} , INCLUDING THE BAYESIAN CHANGEPOINT EXTRACTOR OF APPENDIX II.

Input	
Scene points N / gripper keypoints K	4500 / 4
Channels per point	4 (xyz + gripper indicator)
Sub-goal extraction (BOCPD)	
Feature	end-effector speed (z-scored)
Prior degrees of freedom ν_0	3
Expected segment length / warm-up	15 / 20 steps (Push-T 10 / 8)
Peak threshold / min. spacing	0.03 / 25 steps (Push-T 0.03 / 10)
PointNet++	
Set-abstraction centroids	1024, 512, 256, 128, 64, 16
Grouping radii (m)	0.025/0.05, 0.05/0.1, 0.1/0.2, 0.2/0.4, 0.4/0.8, 0.8/1.6
Neighbours per radius	16, 32
Per-point output	$K \times 3$ displacement + 1 logit
Loss	
Variance ladder Σ	{0.01, 0.05, 0.1, 0.25, 0.5}
Uniform-weight coefficient λ_u	0.1
Training	
Transition sampling p_{tr} / window r	0.5 / 5
Label swap p_{max}	0.5
Optimiser	AdamW, lr 10^{-4} , wd 10^{-5}
LR schedule	cosine, 100 warm-up steps
Gradient clipping	0.3
Batch size / epochs	164 / 100
Demonstrations / validation split	100 / 10% by demonstration

TABLE III
HYPERPARAMETERS SPECIFIC TO THE UNCERTAINTY-AWARE LOW-LEVEL POLICY π_{lo} . ALL OTHERS FOLLOW π_{aux} (TABLE I).

Retained goal candidates J	6
Trunk sequence length	1044 (1024 patch, 8 gripper, 12 goal)
Goal-token time coordinate τ	1.0
Trunk goal prior (content / rank / scale)	1.0 / 1.0 / 0.0
Goal-token MLP (trunk and DiT)	12 $\rightarrow$ 1024 $\rightarrow$ 1024, separate weights
Weighted cross-attention	content scale 1.0, prior scale 1.0
Ground-truth mixing p_{gt} / r / p_{max}	0.5 / 5 / 0.5
Auxiliary goal head	none

APPENDIX IV HARDWARE SETUP

All real-world experiments use a single Franka arm. Demonstrations are collected by teleoperation with GELLO [3], a low-cost leader arm that the operator moves and whose joint configuration the robot follows. Two cameras observe the scene, an Azure Kinect mounted in front of the workspace and a ZED Mini stereo camera mounted on the wrist. Both cameras are calibrated using Zhang’s method [4], giving the intrinsics and extrinsics that let the depth images be unprojected into the robot’s frame. Both stream at 1280×720 , and images are downsampled and randomly cropped to 224×224 for policy learning.

You are labelling SUBGOAL frames in a robot manipulation trajectory, shown as timestamped contact sheets. Each image has its ORIGINAL trajectory frame index printed on it.

{task_context}

WHAT COUNTS AS A SUBGOAL

A subgoal is a frame in which one DISCRETE, VISIBLE event completes. An event is one of the following, and nothing else:

- (a) the gripper fingers close onto an object,
- (b) the gripper fingers open and release an object,
- (c) a held object first loses contact with the surface or support it rested on,
- (d) a held object first makes contact with the surface, container, or fixture it is being placed into or onto,
- (e) an articulated part (drawer, door, lid, handle, button, switch) first reaches its fully opened, fully closed, or fully pressed position,
- (f) the gripper, empty or loaded, first makes contact with the object or fixture it is about to act on.

Every boundary you return must be one of (a)-(f). Before emitting a frame, name which letter it is. If no letter applies, that frame is not a subgoal.

WHAT IS NOT A SUBGOAL

- Moving toward something, however far the arm travels, until contact in (f).
- Carrying an object between two places.
- Lifting or lowering that is already covered by the (c) or (d) frame.
- Retreating, hesitating, regrasping, or small adjustments.
- A frame that describes the same event as a boundary you already emitted -- one event produces exactly ONE boundary, the earliest frame where it is true.
- A frame chosen because a lot of time has passed since the last boundary. Elapsed time is never a reason; only (a)-(f) are.

PRECISION

When an event is visible across several sampled images, choose the FIRST image in which it is unambiguously true. Use only frame indices visibly printed in the contact sheets.

Return structured output only:

{"boundary_indices": [...]}

Fig. 1. The prompt used for vision-language sub-goal extraction. It is task-agnostic apart from the one-sentence task description substituted for {task_context}.

Demonstrations are recorded at 15Hz and rollouts are executed at the same rate.

APPENDIX V PER-TASK DATA COLLECTION

We evaluate on three real-world tasks, pouring, sweeping and pushing. All demonstrations are collected by teleoperating the robot through a complete execution of the task, recorded at 15Hz, and policies are rolled out at the same rate. Images are captured at 1280×720 , then downsampled and randomly cropped to 224×224 . We collect 100 demonstrations per task and keep all of them. Each policy is evaluated on 20 initial scene configurations per task, with objects placed so that the configurations cover the workspace, and the same 20 configurations are used for every method.

For scoring only, we divide each task into stages so that a rollout that fails partway can be credited for the progress it made. These stages play no role in training or inference, where the only supervision is the extracted sub-goal keypoints. In the pouring task the robot pours coffee beans from a jar, and the four stages are picking up the lid and placing it on the table (score 1), grasping the jar (score 2), pouring the beans (score 3), and placing the jar back

(score 4), with a pour that leaves some beans outside the target credited as 3.5. In the sweeping task four crumpled pieces of paper must be pushed into a dustbin, and each piece swept in adds one to the score, up to 4. The pushing task has a single stage, since the robot pushes a box into a designated area in one continuous motion, so Table II of the main paper reports no partial score for it.

APPENDIX VI PROMPT FOR VISION-LANGUAGE SUB-GOAL EXTRACTION

We extract sub-goals with a vision-language model by presenting the demonstration as a sequence of annotated contact sheets. Frames are sampled every 15 steps, with the terminal frame always appended if the stride does not land on it, and each sampled frame is resized to a 224 pixel wide tile and stamped with its original trajectory index. Tiles are laid out five per row and twenty per sheet, giving a 5×4 grid, and consecutive sheets overlap by two sampled frames so that events on a seam remain visible. A demonstration therefore yields roughly one sheet per eighteen sampled frames.

The placeholder {task_context} in Figure 1 is replaced by a one-sentence description of the task. For Kitchen

D1 it reads “A robot arm prepares food in a kitchen: it presses the stove button to turn the stove on, picks up the pot and places it on the stove burner, picks up the bread and puts it in the pot, then moves the pot to the serving region and turns the stove off.” When no instruction is supplied, it falls back to a generic description of a multi-step tabletop manipulation task, so the prompt is otherwise identical across tasks.

The model returns a list of boundary indices, which we intersect with the indices actually shown so that any frame it invents is discarded. If more than 40% of the sampled frames are flagged we treat the response as degenerate and return no boundaries. Surviving boundaries are then refined by re-querying at stride 1 within a ± 15 frame window, and the result is sorted, deduplicated, and merged where predictions fall close together. Two differences from the trajectory-based procedures are worth noting. No gripper signal is merged in, and although the prompt asks the model to recognise finger open and close events visually, the recorded gripper channel is never consulted. We also do not append the terminal frame, so it appears as a boundary only when the model predicts it.

REFERENCES

- [1] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep hierarchical feature learning on point sets in a metric space,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] R. P. Adams and D. J. C. MacKay, “Bayesian online changepoint detection,” *arXiv preprint arXiv:0710.3742*, 2007.
- [3] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel, “GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators,” *arXiv preprint arXiv:2309.13037*, 2023.
- [4] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.